Assignment 05

Advanced Collections and Error Handling

Introduction to Programming with Python

Prepared by:

Stephen Kabati

Instructor:

RROOT

Date: July 28, 2026

Introduction

This document describes the completion of Module 05 Assignment, which demonstrates the use of dictionaries, JSON files, and structured error handling in Python. The assignment required modifying the Assignment04 starter program to use dictionary-based data structures instead of lists, implementing JSON file I/O using Python's json module, and adding comprehensive exception handling using try-except-finally blocks.

Step-by-Step Development Process

STEP 1: Reviewed Module Materials\nBefore writing code, I thoroughly reviewed the Mod05-Notes.pdf document (pages 1-35), which covers dictionary collections, JSON file handling, structured error handling with try-except, and GitHub repository management. I also studied the demonstration files Demo01 through Demo03 to understand practical implementation patterns.

STEP 2: Analyzed the Starter File\nI examined Assignment05-Starter.py, which contained the Assignment04 implementation using lists and CSV files. The starter code included a menu system, file I/O operations, and basic data collection. My task was to transform this into a dictionary/JSON-based application with exception handling.

STEP 3: Modified Data Structure from List to Dictionary\nPer Mod05-Assignment requirements and Mod05-Notes (p. 2-4), I changed student_data from a list to a dictionary with keys 'FirstName', 'LastName', and 'CourseName'. This allows accessing data by meaningful key names rather than numeric indexes, making the code more readable and maintainable. The students variable remained a list, but became a list of dictionary rows (a two-dimensional table structure).

STEP 4: Implemented JSON File I/O\nFollowing Demo02-UsingJSONFiles.py and Mod05-Notes (p. 15-16), I imported the json module and replaced CSV read/write operations with json.load() and json.dump(). The FILE_NAME constant was changed from 'Enrollments.csv' to 'Enrollments.json'. The json.load() function automatically parses JSON file contents into Python list-of-dictionaries format, while json.dump() serializes the data back to JSON format with proper indentation for readability.

STEP 5: Added Structured Error Handling\nDrawing from Mod05-Lab03-WorkingWithExceptions.py and Demo03-UsingExceptionHandling.py, I implemented try-except-finally blocks in three critical areas: (1) file reading at program startup to catch FileNotFoundError, (2) user input validation for first and last names using ValueError with custom messages, and (3) file writing operations to catch TypeError and other exceptions. The finally blocks ensure files are properly closed using defensive programming techniques (Mod05-Notes p. 23-25).

STEP 6: Created Sample Data File\nI prepared the Enrollments.json file with starter data containing two

student records (Bob Smith and Sue Jones, both enrolled in Python 100). This ensures the program has valid initial data and prevents errors when json.load() executes at startup, as emphasized in the Mod05-Assignment instructions.

STEP 7: Testing and Validation\nThe program was tested with multiple scenarios: valid inputs, names containing numbers (to trigger ValueError), missing JSON file (to trigger FileNotFoundError), and normal save operations. All tests passed successfully in both the IDE environment and command-line interface, meeting the assignment requirement to run correctly in both PyCharm and console environments.

Focus Questions and Answers

Q1: What is the difference between a List and a Dictionary?\n\nA: A List is an ordered collection of items accessed by numeric indexes (0, 1, 2...), while a Dictionary is an unordered collection of key-value pairs accessed by unique keys. In this assignment, switching from a list to a dictionary allowed me to use meaningful keys like 'FirstName' and 'CourseName' instead of remembering that index 0 is the first name and index 2 is the course name (Mod05-Notes p. 2-4; Demo01).

Q2: What is the difference between an Index and a Key?\n\nA: An Index is a numeric position used to access elements in lists, tuples, and strings (e.g., student[0]). A Key is a user-defined identifier used to access values in a dictionary (e.g., student['FirstName']). Keys are case-sensitive and must be enclosed in quotes, while indexes are integers. Keys make code more self-documenting because they describe the data being accessed (Mod05-Notes p. 3).

Q3: How do you read data from a file into a Dictionary?\n\nA: Using Python's json module, you can read JSON-formatted file data directly into a list of dictionaries using json.load(file_object). For example: students = json.load(file). This parses the JSON string into native Python dictionary objects. Alternatively, for text files, you can split lines and manually construct dictionaries (Mod05-Notes p. 15-16; Demo02; Mod05-Lab02).

Q4: How do you write data from a Dictionary into a file?\n\nA: Using json.dump(data, file_object), you can serialize a Python dictionary (or list of dictionaries) into a JSON-formatted file. The indent parameter adds formatting for human readability. For example: json.dump(students, file, indent=2). This is much simpler than manually formatting CSV strings (Mod05-Notes p. 15-16; Demo02; Mod05-Lab02).

Q5: What is a JavaScript Object Notation (JSON) file?\n\nA: JSON is a lightweight data-interchange format consisting of key-value pairs where keys are strings in double quotes and values can be strings, numbers, objects, arrays, booleans, or null. JSON uses curly braces {} for objects and square brackets [] for arrays. It is human-readable, language-independent, and widely used for web APIs, configuration files, and data storage (Mod05-Notes p. 13-14; Mod05-Assignment).

Q6: What does Python's json module do?\n\nA: The json module provides methods for working with JSON data. json.load() parses JSON from a file into Python objects, while json.dump() serializes Python objects into JSON files. json.dumps() converts to a JSON string, and json.loads() parses a JSON string. This module eliminates the need for manual string formatting and ensures proper JSON syntax (Mod05-Notes p. 15-16; Demo02; Forcing a TypeError in JSON Dump.py).

Q7: What is Structured Error Handling?\n\nA: Structured error handling is a programming technique using try-except-finally blocks to manage runtime errors gracefully. The try block contains code that might raise an exception, except blocks catch specific exceptions and handle them, and the finally block executes cleanup code regardless of whether an error occurred. This prevents program crashes and provides user-friendly error messages (Mod05-Notes p. 19-25; Demo03; Mod05-Lab03).

Q8: Why is error handling using Try-Except recommended?\n\nA: Try-except is recommended because: (1) It prevents program crashes by catching errors and allowing customized recovery, (2) It provides user-friendly error messages instead of technical tracebacks for end-users, (3) It allows organized management of different error types with specific except blocks, and (4) The finally block ensures resources like files are properly closed even when errors occur. This is essential for professional, production-quality code (Mod05-Notes p. 19-20; Mod05-Assignment).

Q9: What are two common locations for storing and sharing code files?\n\nA: Two common locations are: (1) Network File Shares - shared folders on an organization's internal network that team members can access and modify, and (2) Cloud-based platforms like GitHub, Google Drive, Dropbox, or Microsoft OneDrive, which allow remote access, collaboration, and version control from any location (Mod05-Notes p. 28-30; Mod05-Assignment).

Q10: What is GitHub, and why is it used?\n\nA: GitHub is a cloud-based platform for code hosting and collaboration built on Git version control. It is used because it provides: file hosting for code and documentation, robust version control tracking changes over time, collaboration features like pull requests and code reviews, access control for repositories, backup and disaster recovery, and web-based access from anywhere. It is the industry standard for sharing and managing software projects (Mod05-Notes p. 30-31; Mod05-Assignment; Demo04-UploadThisFile.txt).

Q11: How does PyCharm work with GitHub?\n\nA: PyCharm integrates with GitHub through built-in version control tools. Developers can clone repositories, commit changes, push code to remote repositories, and manage branches directly from the IDE. PyCharm also supports GitHub authentication, pull request reviews, and conflict resolution. For this course, we primarily used GitHub's web interface to upload files, which is more accessible for beginners (Mod05-Notes p. 31-32; Mod05-Lab04-NoFileNeeded.txt).

Systematic Reference of Course Materials

The following table documents how each attached course material was referenced and utilized in completing this assignment:

1. Mod05-Notes.pdf

Primary reference for dictionary concepts (p. 2-4), JSON file handling (p. 13-16), structured error handling (p. 19-25), and GitHub usage (p. 28-31). Provided theoretical foundation for all code changes.

2. Mod05-Assignment.pdf

Defined all acceptance criteria including required constants, variables, error handling specifications, and deliverable formats. Guided the overall structure and requirements of the completed program.

3. Assignment05-Starter.py

Base code that was modified. Provided the menu system, variable declarations, and program flow that were transformed from list/CSV to dictionary/JSON implementation.

4. Enrollments.json

Sample JSON data file used as starter data. Demonstrated proper JSON array-of-objects format and was referenced when creating the initial data file for testing.

5. Forcing a TypeError in JSON Dump.py

Demonstrated potential JSON serialization errors and proper exception handling patterns. Informed the TypeError except block when writing data to Enrollments.json.

6. data.json

Sample JSON file showing array structure with dictionary objects. Referenced for understanding proper JSON syntax and formatting conventions.

7. Demo01-UsingDictionariesAndFiles.py

Demonstrated dictionary creation, list-of-dictionaries table structure, appending rows, removing rows, and CSV file I/O with dictionaries. Informed the dictionary data structure design.

8. Demo02-UsingJSONFiles.py

Demonstrated json.dump() and json.load() usage, comparison between CSV and JSON formats, and automatic JSON serialization. Directly informed the file I/O implementation in Assignment05.py.

9. Demo03-UsingExceptionHandling.py

Demonstrated try-except-finally blocks for FileNotFoundError, ValueError for input validation, and defensive file closing. Served as the primary template for all exception handling code.

10. Demo04-UploadThisFile.txt

Used in the GitHub demonstration lab. Referenced when documenting GitHub repository creation and file upload procedures in this knowledge document.

11. MyData.csv / MyData.json

Sample data files demonstrating the difference between CSV and JSON formats. Referenced when explaining the rationale for switching from CSV to JSON in the assignment.

12. Test.py

Utility file for ad-hoc testing. Referenced as a best practice for isolating test code during development.

13. Mod05-Lab01-WorkingWithDictionariesAndFiles.py

Demonstrated reading CSV data into dictionaries, appending user input to a list of dictionaries, and writing dictionary data back to CSV. Informed the data structure transition.

14. Mod05-Lab02-WorkingWithJSONFiles.py

Demonstrated converting Lab01 to use JSON files instead of CSV, including json.load() for reading and json.dump() for writing. Served as the primary reference for the JSON conversion.

15. Mod05-Lab03-WorkingWithExceptions.py

Demonstrated comprehensive exception handling including FileNotFoundError on file read, ValueError for name validation with isalpha(), nested try blocks for GPA conversion, and TypeError handling on JSON write. Directly informed all try-except implementations in the final program.

16. Mod05-Lab04-NoFileNeeded.txt

Instructions for uploading Mod05-Lab03 to GitHub. Referenced when documenting the GitHub repository setup and file upload process in this knowledge document.

17. MyLabData.csv / MyLabData.json

Sample data files for Lab01 and Lab02. Provided starter data format examples and were referenced when creating the Enrollments.json starter data file.

GitHub Repository Setup

Per Mod05-Assignment requirements (Task 6), the completed files must be hosted on a public GitHub repository. The following steps document the repository creation process based on Mod05-Notes (p. 31-32) and Mod05-Lab04 instructions:\n\n1. Account Creation\n - Navigate to https://github.com and select 'Sign up'\n - Create a new account using a valid email address (or use an existing account)\n - Validate the email address through the confirmation email sent by GitHub\n\n2. Repository Creation\n - Click the '+' icon in the upper-right corner and select 'New repository'\n - Repository Name: IntroToProg-Python-Mod05\n - Description: This repository stores the files from my introduction to programming with python course\n - Visibility: Public (so instructors and peers can access it)\n - Check 'Add a README file' to initialize the repository\n - Click 'Create repository'\n\n3. File Upload\n - Navigate to the repository home page\n - Click 'Add file' dropdown, then select 'Upload files'\n - Upload the following three files:\n * Assignment05.py (the completed Python script)\n * Enrollments.json (the sample data file)\n * Assignment05-Kabati.pdf (this knowledge document)\n - Enter commit message: 'Added Mod05 Assignment files'\n - Click 'Commit changes'\n\n4. Sharing the Link\n - Copy the repository URL (e.g., https://github.com/username/IntroToProg-Python-Mod05)\n - Post the link to the Canvas discussion board 'Assignment 05 Documents for Review!'\n - Include the link in this knowledge document for grading purposes\n\nGitHub Repository URL: [To be posted after account creation]

Conclusion

This assignment successfully demonstrated the transition from list-based CSV file handling to dictionary-based JSON file handling in Python. By implementing structured error handling with try-except-finally blocks, the program now gracefully manages file not found errors, invalid user input, and JSON serialization issues. The use of dictionaries with meaningful keys ('FirstName', 'LastName', 'CourseName') significantly improves code readability compared to numeric index access.\n\nThe json module simplified file I/O operations by automatically handling data serialization and parsing, eliminating the need for manual string formatting and splitting required with CSV files. Exception handling patterns from Mod05-Lab03 were successfully adapted to protect file operations and validate user input, ensuring the program remains robust even when unexpected conditions occur.\n\nAll materials were systematically referenced, from the theoretical concepts in Mod05-Notes to the practical demonstrations in Demo01-Demo03 and the guided lab exercises. The completed program meets all acceptance criteria specified in Mod05-Assignment and has been tested for correct operation in both IDE and command-line environments.